# Frontend Developer Guide: Article Workflow, Photo Upload & Display

This guide explains how the **Author Portal**, **Admin Portal**, and **Public Magazine Website** interact with the **Article Approval & Publishing Workflow**, including **Cover Photos** and **Multi-Photo Galleries**.

---

## 1. End-to-End Article Workflow (What Changed for Frontend)

```
Author Portal (/api/author/article/*)
  ├── Create / Update Article in Tbl_RequestArticle
  │     • Status = "Draft"     (Private to Author; hidden from Admin)
  │     • Status = "Submitted" (Visible in Admin Review Queue)
  │     • Author can Edit, Upload/Remove Photos, or Delete while Status != "Approved"
  │
  ▼
Admin Portal (/api/admin/requestarticle/*)
  ├── Lists & Previews Submitted Articles (with Cover Photo + Gallery Photos)
  ├── Review Decision:
  │     ├── "Rejected" ──► Status = "Rejected" + AdminNotes (Author can edit & resubmit)
  │     └── "Approved" ──► Auto-inserts into Tbl_Article with Status = "Published",
  │                        links all Gallery Photos (Tbl_ArticlePhoto),
  │                        and LOCKS the Author's Tbl_RequestArticle record.
  ▼
Public Website (/api/public/article/*)
  └── Displays Published Articles + Linked Gallery Photos from Tbl_Article
```

### Summary of Key Frontend Changes

- **Author Portal Uses `RequestArticleId` (Not `ArticleId`)**:
- All `/api/author/article/*` endpoints now create, update, detail, submit, and delete records in `Tbl_RequestArticle` using `RequestArticleId` (26-char ULID).
- Once an Admin approves the request, the response includes `PublishedArticleId` (26-char ULID of the live `Tbl_Article`).
- **Author Statuses (`"Draft"` & `"Submitted"`)**:
- Authors can save as `"Draft"` or `"Submitted"` (or `"Submit"`) during `Create` and `Update`, or promote a draft using `POST /api/author/article/SubmitForReview`.
- `"Draft"` articles are **completely hidden from Admin** (`/api/admin/requestarticle/List` excludes `"Draft"`).
- **Edit & Delete Lock Rule in Author UI**:
- While `Status` is `"Draft"`, `"Submitted"`, or `"Rejected"`, the Author **can edit and delete** (`POST /api/author/article/Delete`) the article.
- Once `Status === "Approved"`, the article is **locked**—disable the Edit/Delete buttons in the Author UI.
- **Admin RequestArticle Review Queue (`/api/admin/requestarticle/`)**:*
- `List` and `Detail` now return full article fields (`Title`, `SubTitle`, `Slug`, `Body`, `PhotoUrl`, `CategoryName`, `AuthorName`, `Photos`, `PublishedArticleId`) instead of legacy `PitcherName` / `ProposedTitle`.
- `POST /api/admin/requestarticle/Review` accepts `RequestArticleId`, `Status` (`"Approved"` or `"Rejected"`), `AdminNotes`, `IsFeatured`, and `IsSpotlight`.
- **Public Pitch Endpoint Removed**:
- `POST /api/public/requestarticle/submit` has been removed. Only authenticated Authors can create/submit articles.

---

## 2. How the Photo System Works

Each Article (both staged in `Tbl_RequestArticle` and published in `Tbl_Article`) supports **two types of photos**:

| Photo Type | Database Location | Returned In | Purpose |
|---|---|---|---|
| **1. Cover Photo (Hero)** | `PhotoUrl` (`PhotoCaption`, `PhotoCredit`) | `List` **and** `Detail` | Fast card feed rendering on Home/Category/Admin/Author lists, plus the main hero banner on the Detail page. |
| **2. Gallery Photos (Multi-Photo)** | `Tbl_ArticlePhoto` (`ArticlePhotoId`, `ArticleId`, `RequestArticleId`, `PhotoUrl`, `Caption`, `Credit`, `SortOrder`) | `Detail` (`Photos: [...]`) | Ordered slideshow / editorial photo gallery inside an article. Automatically linked from `RequestArticleId` to `ArticleId` upon Admin approval. |

### File Validation Rules & Limits

| Rule | Limit / Allowed Values | Error Returned if Violated |
|---|---|---|
| **Allowed Extensions** | `.jpg`, `.jpeg`, `.png`, `.webp` | `Invalid file type` |
| **Allowed MIME Types** | `image/jpeg`, `image/png`, `image/webp` | `Invalid file type` |
| **Binary Signature** | File magic bytes must match JPEG, PNG, or RIFF/WEBP | Prevents renamed `.exe` or `.svg` files |
| **Max Single Photo Size** | **5 MB** ( `5,242,880` bytes) per image | `File exceeds 5MB limit` |
| **Max Gallery Photos per Request** | **10 photos** per `Create` or `Update` call | `Maximum 10 gallery photos are allowed` |
| **Max Total Request Body** | **64 MB** total `multipart/form-data` payload | `413 Payload Too Large` |

## 3. Complete API Endpoints Matrix

| Portal | Action | Method & Route | Content-Type | Primary ID Key |
|---|---|---|---|---|
| **Author API** | Create Article ( `Draft` / `Submitted` ) | `POST /api/author/article/Create` | `multipart/form-data` | Returns `RequestArticleId` |
| **Author API** | Update Article ( `Status != Approved` ) | `POST /api/author/article/Update` | `multipart/form-data` | `RequestArticleId` |
| **Author API** | Submit Draft for Admin Review | `POST /api/author/article/SubmitForReview` | `application/json` | `RequestArticleId` |
| **Author API** | Delete My Article ( `Status != Approved` ) | `POST /api/author/article/Delete` | `application/json` | `RequestArticleId` |
| **Author API** | Get My Article Detail (with Gallery) | `POST /api/author/article/Detail` | `application/json` | `RequestArticleId` |
| **Author API** | Get My Article List | `POST /api/author/article/List` | `application/json` | Filter / Paging |
| **Admin API** | Get Submitted Requests List | `POST /api/admin/requestarticle/List` | `application/json` | Filter / Paging ( `Draft` hidden) |
| **Admin API** | Get Submitted Request Detail | `POST /api/admin/requestarticle/Detail` | `application/json` | `RequestArticleId` |
| **Admin API** | Approve / Reject Submitted Request | `POST /api/admin/requestarticle/Review` | `application/json` | `RequestArticleId` |
| **Admin API** | Manage Live Published Articles | `POST /api/admin/article/{Create,Update,Delete,Detail,List,ChangeStatus}` | `multipart` / `json` | `ArticleId` |
| **Public API** | Get Published Article List | `GET /api/public/article/list` | Query Params | — |
| **Public API** | Get Published Article by Slug | `GET /api/public/article/{slug}` | Route Param | `slug` |

## 4. Author Portal Integration ( `/api/author/article/*` )

### 4.1 Author Create Article ( `POST /api/author/article/Create` )

| FormData Key | Type | Required? | Description |
|---|---|---|---|
| `Title` | `string` | ✅ Yes | Article headline (max 200 chars) |
| `SubTitle` | `string` | No | Article subtitle (max 400 chars) |
| `Body` | `string` | No | HTML or Markdown body content |
| `ArticleCategoryId` | `string` | ✅ Yes | 26-character ULID of the category |
| `Status` | `string` | No | `"Draft"` (default) or `"Submitted"` (also accepts `"Submit"` ) |
| `CoverPhoto` | `File` | No | Binary image file ( `.jpg` , `.png` , `.webp` , max 5 MB) |
| `PhotoUrl` | `string` | No | Optional external image URL if `CoverPhoto` file is not uploaded |
| `PhotoCaption` | `string` | No | Cover photo caption (max 200 chars) |
| `PhotoCredit` | `string` | No | Cover photo credit (max 100 chars) |
| `GalleryItems[i].File` | `File` | No | Gallery image file at index `i` ( `0, 1, 2...` ) |
| `GalleryItems[i].Caption` | `string` | No | Caption paired with `GalleryItems[i].File` |
| `GalleryItems[i].Credit` | `string` | No | Credit paired with `GalleryItems[i].File` |
| `GalleryItems[i].SortOrder` | `number` | No | Display order ( `1, 2, 3...` ) paired with `GalleryItems[i].File` |

```ts
export interface GalleryUploadItem {
  file: File;
  caption?: string;
  credit?: string;
  sortOrder: number;
}

export interface AuthorCreateArticleInput {
  title: string;
  subTitle?: string;
  body?: string;
  articleCategoryId: string;
  status?: "Draft" | "Submitted";
  coverPhoto?: File | null;
  photoUrl?: string;
  photoCaption?: string;
  photoCredit?: string;
  galleryItems?: GalleryUploadItem[];
}

export async function authorCreateArticle(
  authorApiBaseUrl: string,
  input: AuthorCreateArticleInput,
  jwtToken?: string
) {
  const formData = new FormData();

  formData.append("Title", input.title);
  if (input.subTitle) formData.append("SubTitle", input.subTitle);
  if (input.body) formData.append("Body", input.body);
  formData.append("ArticleCategoryId", input.articleCategoryId);
  formData.append("Status", input.status ?? "Draft");

  if (input.coverPhoto) {
    formData.append("CoverPhoto", input.coverPhoto);
  } else if (input.photoUrl) {
    formData.append("PhotoUrl", input.photoUrl);
  }
  if (input.photoCaption) formData.append("PhotoCaption", input.photoCaption);
  if (input.photoCredit) formData.append("PhotoCredit", input.photoCredit);

  input.galleryItems?.forEach((item, index) => {
    formData.append(`GalleryItems[${index}].File`, item.file);
    formData.append(`GalleryItems[${index}].Caption`, item.caption ?? "");
    formData.append(`GalleryItems[${index}].Credit`, item.credit ?? "");
    formData.append(`GalleryItems[${index}].SortOrder`, String(item.sortOrder));
  });

  const response = await fetch(`${authorApiBaseUrl}/api/author/article/Create`, {
    method: "POST",
    headers: jwtToken ? { Authorization: `Bearer ${jwtToken}` } : undefined,
    body: formData,
  });

  return await response.json();
}
```

### 4.2 Author Update Article ( `POST /api/author/article/Update` )

> **Important**: Use `RequestArticleId` (not `ArticleId` ) when updating an article in the Author Portal.

| FormData Key | Type | Required? | Description |
|---|---|---|---|
| `RequestArticleId` | `string` | ✅ Yes | 26-character ULID of the staged article in `Tbl_RequestArticle` |
| `Title` | `string` | ✅ Yes | Article headline |
| `SubTitle` | `string` | No | Article subtitle |
| `Body` | `string` | No | HTML or Markdown body content |
| `ArticleCategoryId` | `string` | ✅ Yes | 26-character ULID of the category |
| `Status` | `string` | No | `"Draft"` or `"Submitted"` (editable while current status is not `"Approved"`) |
| `CoverPhoto` | `File` | No | Upload a new file here **only** if replacing the cover photo |
| `PhotoUrl` | `string` | No | Pass existing `PhotoUrl` here to keep the current cover photo |
| `PhotoCaption` | `string` | No | Cover photo caption |
| `PhotoCredit` | `string` | No | Cover photo credit |
| `NewGalleryItems[i].File` | `File` | No | New gallery image file to add (`0, 1, 2...`) |
| `NewGalleryItems[i].Caption` | `string` | No | Caption for the new gallery image |
| `NewGalleryItems[i].Credit` | `string` | No | Credit for the new gallery image |
| `NewGalleryItems[i].SortOrder` | `number` | No | SortOrder for the new gallery image |
| `RemoveArticlePhotoIds[i]` | `string` | No | 26-char `ArticlePhotoId` ULID of an existing gallery photo to delete |

```ts
export interface AuthorUpdateArticleInput {
  requestArticleId: string;
  title: string;
  subTitle?: string;
  body?: string;
  articleCategoryId: string;
  status?: "Draft" | "Submitted";
  newCoverPhoto?: File | null;
  existingPhotoUrl?: string;
  photoCaption?: string;
  photoCredit?: string;
  newGalleryItems?: GalleryUploadItem[];
  removeArticlePhotoIds?: string[];
}

export async function authorUpdateArticle(
  authorApiBaseUrl: string,
  input: AuthorUpdateArticleInput,
  jwtToken?: string
) {
  const formData = new FormData();

  formData.append("RequestArticleId", input.requestArticleId);
  formData.append("Title", input.title);
  if (input.subTitle) formData.append("SubTitle", input.subTitle);
  if (input.body) formData.append("Body", input.body);
  formData.append("ArticleCategoryId", input.articleCategoryId);
  if (input.status) formData.append("Status", input.status);

  if (input.newCoverPhoto) {
    formData.append("CoverPhoto", input.newCoverPhoto);
  } else if (input.existingPhotoUrl) {
    formData.append("PhotoUrl", input.existingPhotoUrl);
  }
  if (input.photoCaption) formData.append("PhotoCaption", input.photoCaption);
  if (input.photoCredit) formData.append("PhotoCredit", input.photoCredit);

  input.newGalleryItems?.forEach((item, index) => {
    formData.append(`NewGalleryItems[${index}].File`, item.file);
    formData.append(`NewGalleryItems[${index}].Caption`, item.caption ?? "");
    formData.append(`NewGalleryItems[${index}].Credit`, item.credit ?? "");
    formData.append(`NewGalleryItems[${index}].SortOrder`, String(item.sortOrder));
  });

  input.removeArticlePhotoIds?.forEach((photoId, index) => {
    formData.append(`RemoveArticlePhotoIds[${index}]`, photoId);
  });

  const response = await fetch(`${authorApiBaseUrl}/api/author/article/Update`, {
    method: "POST",
    headers: jwtToken ? { Authorization: `Bearer ${jwtToken}` } : undefined,
    body: formData,
  });

  return await response.json();
}
```

### 4.3 Author Detail, SubmitForReview & Delete ( `application/json` )

```ts
// POST /api/author/article/Detail
// POST /api/author/article/SubmitForReview
// POST /api/author/article/Delete
// Request Body:
{
  "RequestArticleId": "01JREQ00000000000000000001"
}
```

## 5. Admin Portal Review Queue Integration ( `/api/admin/requestarticle/*` )

### 5.1 Admin List Submitted Requests ( `POST /api/admin/requestarticle/List` )

```
{
  "Page": 1,
  "PageSize": 20,
  "SearchKeyword": null,
  "ArticleCategoryId": null,
  "AuthorId": null,
  "Status": "Submitted"
}
```

> **Note**: Pass `"Status": null` to view all non-draft requests ( `Submitted` , `Approved` , `Rejected` ), or filter by `"Submitted"` , `"Approved"` , or `"Rejected"` . Author `"Draft"` items are never returned to Admin.

### 5.2 Admin Get Request Detail ( `POST /api/admin/requestarticle/Detail` )

**Request Body:**

```
{
  "RequestArticleId": "01JREQ00000000000000000001"
}
```

**Response Body ( `RequestArticleResModel` ):**

```
{
  "IsSuccess": true,
  "IsError": false,
  "RespType": "Success",
  "RespCode": "MS#000",
  "RespDesp": "Success",
  "Data": {
    "RequestArticleId": "01JREQ00000000000000000001",
    "Title": "Monastic Architecture in Contemporary Homes",
    "SubTitle": "How Cistercian simplicity inspires serene urban residences",
    "Slug": "monastic-architecture-in-contemporary-homes",
    "Body": "<p>An essay examining how contemporary architects...</p>",
    "PhotoUrl": "https://images.unsplash.com/photo-1513694203232-719a280e022f",
    "PhotoCaption": "Minimalist limestone courtyard in Kyoto.",
    "PhotoCredit": "Photo by Kenji Sato",
    "ArticleCategoryId": "01JCAT00000000000000000003",
    "CategoryName": "Musing",
    "AuthorId": "01JAUTHOR0000000000000001",
    "AuthorName": "Elena Rostova",
    "Status": "Submitted",
    "SubmittedAt": "2026-10-11T01:22:41+06:30",
    "ReviewedAt": null,
    "ReviewedByAdminId": null,
    "AdminNotes": null,
    "PublishedArticleId": null,
    "CreatedAt": "2026-10-11T01:22:41+06:30",
    "UpdatedAt": null,
    "Photos": [
      {
        "ArticlePhotoId": "01JAPHT0000000000000000003",
        "ArticleId": null,
        "RequestArticleId": "01JREQ00000000000000000001",
        "PhotoUrl": "https://images.unsplash.com/photo-1600585154526-990dced4db0d",
        "Caption": "Raw concrete and warm oak timber framing.",
        "Credit": "Photo by Kenji Sato",
        "SortOrder": 1,
        "CreatedAt": "2026-10-11T01:22:41+06:30"
      }
    ]
  }
}
```

### 5.3 Admin Approve or Reject ( `POST /api/admin/requestarticle/Review` )

**Request Body:**

```
{
  "RequestArticleId": "01JREQ00000000000000000001",
  "Status": "Approved",
  "AdminNotes": "Approved for immediate publication.",
  "IsFeatured": true,
  "IsSpotlight": false
}
```

- **`Status: "Approved"`** : Creates the live article in `Tbl_Article` with `Status = "Published"`, populates `ArticleId` on all attached `Photos`, locks the `Tbl_RequestArticle` record, and returns `PublishedArticleId` in `Data`.
- **`Status: "Rejected"`** : Sets `Status = "Rejected"` and stores `AdminNotes` so the Author can revise and resubmit.

---

## 6. Displaying Photos on the Frontend ( `resolveImageUrl` )

Because `PhotoUrl` can be either:

- A relative path uploaded to our server ( `/uploads/articles/2026/10/01J...webp` ), **or**
- An external URL ( `https://images.unsplash.com/...` ) from seeded/external images,

use this helper everywhere in your Frontend components:

```
const API_BASE_URL = process.env.NEXT_PUBLIC_API_BASE_URL ?? "http://localhost:5222";

export function resolveImageUrl(
  photoUrl: string | null | undefined,
  fallbackUrl = "/images/placeholder-editorial.jpg"
): string {
  if (!photoUrl || photoUrl.trim() === "") {
    return fallbackUrl;
  }

  if (
    photoUrl.startsWith("http://") ||
    photoUrl.startsWith("https://") ||
    photoUrl.startsWith("blob:") ||
    photoUrl.startsWith("data:")
  ) {
    return photoUrl;
  }

  const normalizedBase = API_BASE_URL.replace(/\/+$/, "");
  const normalizedPath = photoUrl.startsWith("/") ? photoUrl : `/${photoUrl}`;
  return `${normalizedBase}${normalizedPath}`;
}
```

---

## 7. Checklist & Common Pitfalls for Frontend Developers

- **PascalCase JSON Response Keys**:
- All backend APIs serialize JSON responses with **PascalCase** property names ( `IsSuccess` , `RespCode` , `Data` , `RequestArticleId` , `ArticleId` , `PhotoUrl` , `Photos` ).
- **Author Portal vs Admin Article Portal IDs**:
- Author Portal ( `/api/author/article/` ) *and Admin Review Queue (* `/api/admin/requestarticle/` ) use **`RequestArticleId`** .
- Admin Published Articles ( `/api/admin/article/` ) *and Public Website (* `/api/public/article/` ) use **`ArticleId`** / **`Slug`** .
- **Disable Edit/Delete in Author UI When `Status === "Approved"`** :
- Once an article has `Status === "Approved"` , the backend rejects any Author `Update` or `Delete` call. Show a `"Published"` badge and optionally link to the live article via `PublishedArticleId` or `Slug` .
- **Do NOT manually set `Content-Type` header when sending `FormData`** :
- Let `fetch` or `axios` set `Content-Type: multipart/form-data; boundary=...` automatically.